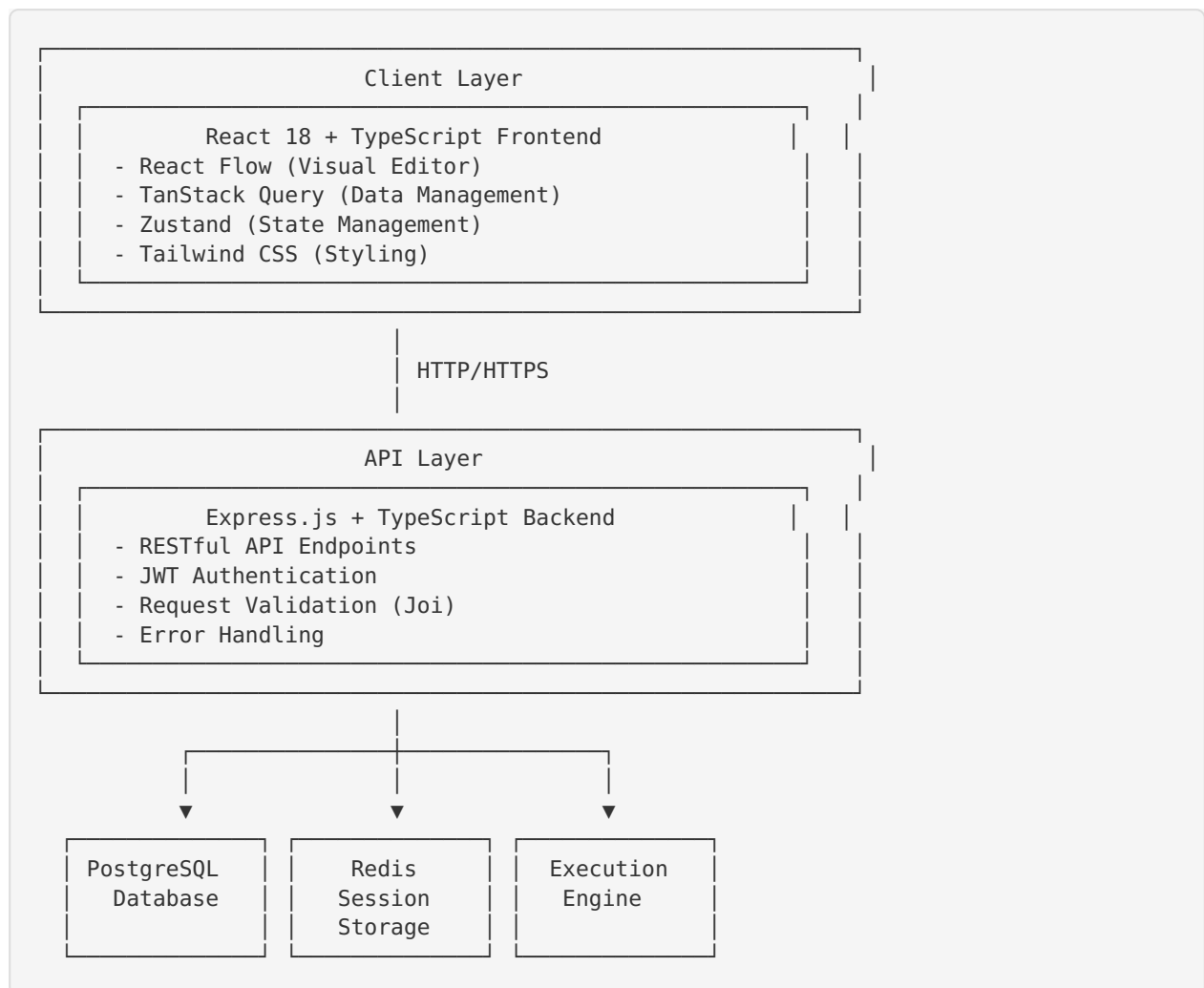


SephiroFlows Architecture - Phase 1 MVP

Overview

SephiroFlows Phase 1 MVP is built with a modern, scalable architecture designed for future expansion while maintaining simplicity in the initial implementation.

System Architecture



Component Details

Frontend Architecture

React Application Structure

```
src/
├── components/           # Reusable UI components
│   ├── Layout.tsx       # Main layout with sidebar
│   ├── PrivateRoute.tsx # Auth guard component
│   ├── NodeLibrary.tsx  # Node selection panel
│   └── NodeConfigPanel.tsx # Node configuration
├── pages/               # Page components
│   ├── LoginPage.tsx
│   ├── RegisterPage.tsx
│   ├── DashboardPage.tsx
│   ├── WorkflowsPage.tsx
│   ├── WorkflowEditorPage.tsx
│   ├── ExecutionsPage.tsx
│   └── ExecutionDetailPage.tsx
├── services/            # API clients
│   └── api.ts           # Axios-based API client
├── stores/              # State management (Zustand)
│   ├── authStore.ts     # Authentication state
│   └── workflowStore.ts  # Workflow editor state
└── types/               # TypeScript definitions
    └── index.ts         # Shared types
```

State Management Strategy

1. **Local State:** React useState/useReducer for component-specific state
2. **Global State:** Zustand stores for auth and workflow editor
3. **Server State:** TanStack Query for API data caching

Data Flow

1. User interacts with UI
2. Component triggers API call via TanStack Query
3. API client sends HTTP request with JWT token
4. Response cached by TanStack Query
5. Component re-renders with new data

Backend Architecture

Express Application Structure

```

src/
├── config/                # Configuration
│   ├── database.ts       # PostgreSQL connection
│   └── redis.ts          # Redis connection
├── controllers/          # Request handlers
│   ├── authController.ts
│   ├── workflowController.ts
│   ├── executionController.ts
│   └── nodeController.ts
├── middleware/           # Express middleware
│   ├── auth.ts           # JWT authentication
│   ├── validation.ts     # Request validation
│   └── errorHandler.ts  # Global error handling
├── routes/               # API routes
│   ├── auth.ts
│   ├── workflows.ts
│   ├── executions.ts
│   └── nodes.ts
├── engine/               # Workflow execution
│   ├── WorkflowExecutor.ts # Main executor
│   ├── NodeExecutor.ts    # Node dispatcher
│   └── ExpressionEvaluator.ts # Expression parser
├── nodes/                # Built-in nodes
│   ├── ManualTriggerNode.ts
│   ├── HttpRequestNode.ts
│   ├── DataTransformNode.ts
│   ├── ConditionalNode.ts
│   ├── SetVariableNode.ts
│   └── SendEmailNode.ts
├── types/                # TypeScript definitions
│   └── index.ts          # Shared types

```

Request Flow

1. HTTP request received by Express
2. CORS middleware validates origin
3. Body parser middleware parses JSON
4. Route handler matched
5. Authentication middleware validates JWT
6. Validation middleware checks request body
7. Controller executes business logic
8. Response sent to client
9. Error handler catches any errors

Database Architecture

PostgreSQL Schema

Core Tables:

- `companies` - Tenant/company records
- `users` - User accounts
- `workflows` - Workflow definitions
- `workflow_runs` - Execution records
- `node_runs` - Individual node execution logs
- `node_definitions` - Available node types
- `credentials` - Encrypted credentials (future)

Key Relationships:

- Users belong to Companies (many-to-one)
- Workflows belong to Companies (many-to-one)
- Workflows created by Users (many-to-one)
- WorkflowRuns belong to Workflows (many-to-one)
- NodeRuns belong to WorkflowRuns (many-to-one)

Indexing Strategy:

- Primary keys: UUID with `gen_random_uuid()`
- Foreign keys: Indexed for JOIN performance
- Status fields: Indexed for filtering
- Timestamp fields: Indexed for sorting
- Compound indexes: `(company_id, status)` for common queries

Execution Engine

Workflow Execution Flow

1. User triggers workflow
↓
2. WorkflowRun record created (status: queued)
↓
3. WorkflowExecutor.execute() called
↓
4. Parse workflow graph (nodes + edges)
↓
5. Topological sort for execution order
↓
6. For each node in order:
 - a. Create NodeRun record (status: running)
 - b. Evaluate expressions in parameters
 - c. Execute node via NodeExecutor
 - d. Store output in context
 - e. Update NodeRun (status: success/failed)
 ↓
7. Update WorkflowRun (status: success/failed)
↓
8. Return execution results

Expression Evaluation

The ExpressionEvaluator processes dynamic values:

```
// Input: "{{${node.http1.data.status}}}"
// Process:
1. Detect expression pattern {{...}}
2. Parse expression type ($node, $trigger, $variables)
3. Resolve value from execution context
4. Replace expression with actual value
// Output: 200
```

Supported Expressions:

- {{\${node.nodeId.data.field}}} - Node output
- {{\${trigger.data.field}}} - Trigger data
- {{\${variables.name}}} - Workflow variables
- {{\${now()}}} - Built-in functions

Authentication & Security

JWT Authentication Flow

1. User submits **login** credentials
↓
2. Backend **validates** email/password
↓
3. Generate session ID
↓
4. Store session in Redis (24hr TTL)
↓
5. Generate JWT **token** with session ID
↓
6. **Return token to** client
↓
7. Client stores **token** in localStorage
↓
8. Client includes **token** in Authorization header
↓
9. Backend **validates token and** checks Redis session

Security Measures

- **Password Hashing:** bcrypt with 10 rounds
- **JWT Secret:** 256-bit minimum
- **Session Storage:** Redis with TTL
- **CORS:** Configured for frontend origin
- **SQL Injection:** Parameterized queries
- **XSS Protection:** React's built-in escaping
- **Credentials:** AES-256-GCM encryption

Data Models

Workflow Model

```
interface Workflow {
  id: string;
  company_id: string;
  name: string;
  description?: string;
  created_by: string;
  status: 'draft' | 'active' | 'paused';
  nodes: WorkflowNode[]; // JSON array
  edges: WorkflowEdge[]; // JSON array
  settings: any;           // JSON object
  created_at: Date;
  updated_at: Date;
}
```

Node Model

```
interface WorkflowNode {
  id: string;           // Unique node instance ID
  type: string;         // Node type (e.g., 'action.http')
  position: {           // Canvas position
    x: number;
    y: number;
  };
  data: {
    label: string;      // Display name
    parameters: any;    // Node-specific config
    credentialId?: string;
  };
}
```

Execution Context

```
interface ExecutionContext {
  workflowRunId: string;
  companyId: string;
  userId?: string;
  variables: Record<string, any>;
  nodeOutputs: Record<string, any>;
  triggerData: any;
}
```

Scalability Considerations

Current Limitations (Phase 1)

- Synchronous execution (blocks API thread)
- Single database instance
- No horizontal scaling
- No queue system
- Limited concurrent executions

Future Improvements (Phase 2+)

- **BullMQ:** Async job queue for executions
- **Worker Pool:** Separate execution workers
- **Database Replication:** Read replicas
- **Caching Layer:** Redis for query results
- **Load Balancing:** Multiple API instances
- **WebSocket:** Real-time execution updates

Performance Optimization

Database

- Connection pooling (20 max connections)
- Indexed queries
- Efficient JSON operations

API

- Response compression (gzip)
- Request validation
- Error caching

Frontend

- Code splitting
- Lazy loading
- React Query caching
- Optimistic updates

Monitoring & Logging

Current Implementation

- Console logging (development)
- Morgan HTTP logger
- Error stack traces
- Execution logs in database

Future Implementation

- Structured logging (Winston)
- Log aggregation (Loki)
- Metrics (Prometheus)
- Dashboards (Grafana)
- Alerts (AlertManager)

Deployment Architecture

Development

```
Localhost:  
- Frontend: localhost:5173  
- Backend: localhost:3000  
- PostgreSQL: localhost:5432  
- Redis: localhost:6379
```

Production (Future)

```
Cloud Infrastructure:  
- Frontend: CDN (Cloudflare, Vercel)  
- Backend: Container orchestration (Kubernetes)  
- Database: Managed PostgreSQL (AWS RDS)  
- Redis: Managed Redis (AWS ElastiCache)  
- Load Balancer: nginx/AWS ALB
```

Technology Decisions

Why Express.js?

- Simple and lightweight
- Large ecosystem
- Easy to extend
- Well-documented

Why PostgreSQL?

- JSONB support for flexible schemas
- Strong consistency
- Advanced indexing
- Mature and reliable

Why React Flow?

- Built for node-based editors
- Excellent performance
- Active development
- Rich feature set

Why Zustand?

- Simple API
- No boilerplate
- TypeScript support
- Minimal bundle size

Why TanStack Query?

- Automatic caching
- Background refetching
- Optimistic updates

- Excellent DevTools

Conclusion

Phase 1 MVP provides a solid foundation with:

- Clean architecture
- Type safety (TypeScript)
- Scalable design patterns
- Security best practices
- Modern tech stack
- Room for future growth

The architecture is designed to evolve incrementally without major rewrites as we add features in future phases.